SAÉ S3.A.01 - Développement d'une Application

Sprint 1 - Représentation des nombres complexes

Romain Wallon

IUT de Lens - BUT 2 Informatique

2026-2027

UNIVERSITÉ D'ARTOIS

IUT Institut Universitaire Technologie LENS

Sujet complet disponible

Le sujet avec tous les détails est maintenant disponible, et en particulier la répartition des *sprints*

Sujet complet disponible

Le sujet avec tous les détails est maintenant disponible, et en particulier la répartition des *sprints*

Vous y retrouverez tous les patrons attendus, et les classes nécessaires

Sujet complet disponible

Le sujet avec tous les détails est maintenant disponible, et en particulier la répartition des *sprints*

Vous y retrouverez tous les patrons attendus, et les classes nécessaires

Cependant, une certaine liberté vous est laissée sur la manière de les implémenter

Sujet complet disponible

Le sujet avec tous les détails est maintenant disponible, et en particulier la répartition des *sprints*

Vous y retrouverez tous les patrons attendus, et les classes nécessaires

Cependant, une certaine liberté vous est laissée sur la manière de les implémenter

Notez que la solution proposée n'est pas forcément la seule, et que les différences avec votre solution de ce matin ne la rende pas incorrecte

Factorisation du code

Pendant que vous implémenterez votre application, évitez au maximum la duplication de code

Factorisation du code

Pendant que vous implémenterez votre application, évitez au maximum la duplication de code

Si vous vous apercevez que vous répétez du code, demandez-vous s'il ne peut pas être placé dans une méthode ou dans une classe parente (potentiellement abstraite)

Factorisation du code

Pendant que vous implémenterez votre application, évitez au maximum la duplication de code

Si vous vous apercevez que vous répétez du code, demandez-vous s'il ne peut pas être placé dans une méthode ou dans une classe parente (potentiellement abstraite)

Vous pouvez vous en occuper à la fin d'un sprint

D'autres bonnes pratiques

D'autres bonnes pratiques

Évitez les `if` ! Faites du polymorphisme !

D'autres bonnes pratiques

Évitez les `if` ! Faites du polymorphisme !

Si vous constatez que des classes n'ont pas d'état interne, demandez-vous s'il n'est pas possible d'en faire des singletons

D'autres bonnes pratiques

Évitez les `if` ! Faites du polymorphisme !

Si vous constatez que des classes n'ont pas d'état interne, demandez-vous s'il n'est pas possible d'en faire des singletons

Faites confiance à SonarQube !

Écriture de tests

Vous devez écrire des tests pour chacune des fonctionnalités que vous allez développer, en plus des essais manuels que vous effectuerez

Écriture de tests

Vous devez écrire des tests pour chacune des fonctionnalités que vous allez développer, en plus des essais manuels que vous effectuerez

Le but à atteindre est d'avoir une couverture de code autour de 80%

Écriture de tests

Vous devez écrire des tests pour chacune des fonctionnalités que vous allez développer, en plus des essais manuels que vous effectuerez

Le but à atteindre est d'avoir une couverture de code autour de 80%

Le projet est configuré pour utiliser JUnit 5 et JaCoCo, ce qui vous permettra d'avoir facilement des rapports de couverture

Écriture de tests

Vous devez écrire des tests pour chacune des fonctionnalités que vous allez développer, en plus des essais manuels que vous effectuerez

Le but à atteindre est d'avoir une couverture de code autour de 80%

Le projet est configuré pour utiliser JUnit 5 et JaCoCo, ce qui vous permettra d'avoir facilement des rapports de couverture

Utilisez des cas de base simples pour tester vos fonctionnalités

Écriture de tests

Vous devez écrire des tests pour chacune des fonctionnalités que vous allez développer, en plus des essais manuels que vous effectuerez

Le but à atteindre est d'avoir une couverture de code autour de 80%

Le projet est configuré pour utiliser JUnit 5 et JaCoCo, ce qui vous permettra d'avoir facilement des rapports de couverture

Utilisez des cas de base simples pour tester vos fonctionnalités

Aujourd'hui, les tests vous sont déjà fournis, mais vous devrez peut-être les compléter pour atteindre le seuil de 80% de couverture de code

But du *sprint*

Dans ce *sprint*, vous devez implémenter les classes permettant de manipuler les nombres complexes, de manière à ce que les tests unitaires fournis passent correctement

But du *sprint*

Dans ce *sprint*, vous devez implémenter les classes permettant de manipuler les nombres complexes, de manière à ce que les tests unitaires fournis passent correctement

Ces implémentations seront la base de toute la suite de la SAÉ, alors assurez-vous de bien respecter les consignes et les tests fournis

Opérations sur les nombres complexes

Un nombre complexe est un nombre z qui peut s'écrire sous la forme $z = a + ib$, où a et b sont des nombres réels et i est un nombre dit *imaginaire* tel que $i^2 = -1$

Opérations sur les nombres complexes

Un nombre complexe est un nombre z qui peut s'écrire sous la forme $z = a + ib$, où a et b sont des nombres réels et i est un nombre dit *imaginaire* tel que $i^2 = -1$

Le nombre $a = \mathfrak{Re}(z)$ est appelé partie réelle de z, et le nombre $b = \mathfrak{Im}(z)$ est appelé partie imaginaire de z

Opérations sur les nombres complexes

Un nombre complexe est un nombre z qui peut s'écrire sous la forme $z = a + ib$, où a et b sont des nombres réels et i est un nombre dit *imaginaire* tel que $i^2 = -1$

Le nombre $a = \mathfrak{Re}(z)$ est appelé partie réelle de z, et le nombre $b = \mathfrak{Im}(z)$ est appelé partie imaginaire de z

Vous devez implémenter l'interface `IComplex` qui définit les opérations de base sur les nombres complexes

Opérations sur les nombres complexes

Un nombre complexe est un nombre z qui peut s'écrire sous la forme $z = a + ib$, où a et b sont des nombres réels et i est un nombre dit *imaginaire* tel que $i^2 = -1$

Le nombre $a = \mathfrak{Re}(z)$ est appelé partie réelle de z, et le nombre $b = \mathfrak{Im}(z)$ est appelé partie imaginaire de z

Vous devez implémenter l'interface `IComplex` qui définit les opérations de base sur les nombres complexes

Il suffit essentiellement d'implémenter les formules données dans le sujet dans les méthodes appropriées

Plan complexe

Les nombres complexes peuvent être représentés dans un plan à deux dimensions, appelé plan complexe, où l'axe horizontal représente la partie réelle et l'axe vertical représente la partie imaginaire

Plan complexe

Les nombres complexes peuvent être représentés dans un plan à deux dimensions, appelé plan complexe, où l'axe horizontal représente la partie réelle et l'axe vertical représente la partie imaginaire

Pour nos fractales, nous aurons besoin de faire le lien entre un pixel `(row, column)` de l'image et un nombre complexe $z = a + ib$ à l'aide des formules :

- $a = \texttt{column} - \frac{\texttt{width}}{2}$
- $b = \frac{\texttt{height}}{2} - \texttt{row}$

Plan complexe

Les nombres complexes peuvent être représentés dans un plan à deux dimensions, appelé plan complexe, où l'axe horizontal représente la partie réelle et l'axe vertical représente la partie imaginaire

Pour nos fractales, nous aurons besoin de faire le lien entre un pixel `(row, column)` de l'image et un nombre complexe $z = a + ib$ à l'aide des formules :

- $a = \texttt{column} - \frac{\texttt{width}}{2}$
- $b = \frac{\texttt{height}}{2} - \texttt{row}$

Veillez à définir les bons attributs dans votre implémentation de l'interface `IComplexPlane` pour pouvoir effectuer ces calculs

Suites complexes

Nous allons considérer des suites récurrentes de nombres complexes, définies par une fonction f telle que $z_{n+1} = f(z_n)$

Suites complexes

Nous allons considérer des suites récurrentes de nombres complexes, définies par une fonction f telle que $z_{n+1} = f(z_n)$

Pour l'instant, nous allons uniquement considérer la suite de Julia définie par la fonction suivante (à utiliser dans la méthode `computeNext`) :

$$f(z) = z^2 + c$$

Suites complexes

Nous allons considérer des suites récurrentes de nombres complexes, définies par une fonction f telle que $z_{n+1} = f(z_n)$

Pour l'instant, nous allons uniquement considérer la suite de Julia définie par la fonction suivante (à utiliser dans la méthode `computeNext`) :

$$f(z) = z^2 + c$$

Ici, c est une constante (par exemple, $c = -0.4 + 0.6i$) et z_0 est le nombre complexe pour lequel la suite est évaluée

Suites complexes

Nous allons considérer des suites récurrentes de nombres complexes, définies par une fonction f telle que $z_{n+1} = f(z_n)$

Pour l'instant, nous allons uniquement considérer la suite de Julia définie par la fonction suivante (à utiliser dans la méthode `computeNext`) :

$$f(z) = z^2 + c$$

Ici, c est une constante (par exemple, $c = -0.4 + 0.6i$) et z_0 est le nombre complexe pour lequel la suite est évaluée

Plus tard, z_0 correspondra à un pixel de l'image

Implémenter `Iterable<IComplex>`

Pour itérer sur les termes d'une suite, vous devez implémenter l'interface `Iterable` (qui est déjà étendue par l'interface `IComplexSequence`)

Implémenter `Iterable<IComplex>`

Pour itérer sur les termes d'une suite, vous devez implémenter l'interface `Iterable` (qui est déjà étendue par l'interface `IComplexSequence`)

Vous devez donc retourner un objet `Iterator<IComplex>` dans la méthode `iterator()` de votre suite

Implémenter `Iterable<IComplex>`

Pour itérer sur les termes d'une suite, vous devez implémenter l'interface `Iterable` (qui est déjà étendue par l'interface `IComplexSequence`)

Vous devez donc retourner un objet `Iterator<IComplex>` dans la méthode `iterator()` de votre suite

Il s'agit d'un nouvel objet à créer, à partir des informations stockées dans la suite

Implémenter `Iterator<IComplex>`

L'itérateur doit calculer les termes de la suite à partir du nombre complexe initial z_0

Implémenter `Iterator<IComplex>`

L'itérateur doit calculer les termes de la suite à partir du nombre complexe initial z_0

La méthode `hasNext()` doit retourner `true` lorsque le nombre de termes calculés jusqu'à présent est inférieur au nombre maximal d'itérations, et que le module du terme courant est inférieur ou égal à 2, et `false` sinon

Implémenter `Iterator<IComplex>`

L'itérateur doit calculer les termes de la suite à partir du nombre complexe initial z_0

La méthode `hasNext()` doit retourner `true` lorsque le nombre de termes calculés jusqu'à présent est inférieur au nombre maximal d'itérations, et que le module du terme courant est inférieur ou égal à 2, et `false` sinon

La méthode `next()` doit calculer le prochain terme de la suite à partir du terme précédent, et le retourner

Implémenter `Iterator<IComplex>`

L'itérateur doit calculer les termes de la suite à partir du nombre complexe initial z_0

La méthode `hasNext()` doit retourner `true` lorsque le nombre de termes calculés jusqu'à présent est inférieur au nombre maximal d'itérations, et que le module du terme courant est inférieur ou égal à 2, et `false` sinon

La méthode `next()` doit calculer le prochain terme de la suite à partir du terme précédent, et le retourner

Vous ne devez calculer qu'un seul terme à la fois dans cette méthode !

Implémenter `Iterator<IComplex>`

L'itérateur doit calculer les termes de la suite à partir du nombre complexe initial z_0

La méthode `hasNext()` doit retourner `true` lorsque le nombre de termes calculés jusqu'à présent est inférieur au nombre maximal d'itérations, et que le module du terme courant est inférieur ou égal à 2, et `false` sinon

La méthode `next()` doit calculer le prochain terme de la suite à partir du terme précédent, et le retourner

Vous ne devez calculer qu'un seul terme à la fois dans cette méthode !

Veillez à utiliser uniquement l'interface `IComplexSequence` pour calculer les termes de la suite, afin de pouvoir réutiliser votre itérateur pour d'autres suites complexes

Pour en savoir plus

Le sujet de la SAÉ reprend l'essentiel des informations données ici

Pour en savoir plus

Le sujet de la SAÉ reprend l'essentiel des informations données ici

Un lien vers une vidéo en lien avec le sujet est aussi disponible sur *Moodle* pour vous aider à mieux comprendre les concepts abordés dans cette SAÉ

Démonstration finale

À l'issue de ce *sprint*, votre bibliothèque doit passer les tests fournis dans les classes de test `ComplexTest`, `ComplexPlaneTest` et `ComplexSequenceTest`, et atteindre un seuil de couverture de code d'au moins 80%

Attendus de ce *sprint*

- *Fork* du dépôt GitLab fourni et ajout des membres du groupe
- Mise à jour du fichier `README.md`
- Import du projet dans vos IDE avant 14h00

Attendus de ce *sprint*

- *Fork* du dépôt GitLab fourni et ajout des membres du groupe
- Mise à jour du fichier `README.md`
- Import du projet dans vos IDE avant 14h00

- Représentation d'un nombre complexe
- Représentation du plan complexe permettant de faire la relation entre les pixels d'une image et un nombre complexe
- Représentation d'une suite de nombres complexes itérable

Validations techniques

- Implémentation de l'interface `IComplexPlane`
- Implémentation de l'interface `IComplexSequence` pour la suite de Julia
- Implémentation de l'itérateur pour les suites

Validations techniques

- Implémentation de l'interface `IComplexPlane`
- Implémentation de l'interface `IComplexSequence` pour la suite de Julia
- Implémentation de l'itérateur pour les suites

N'attendez pas la dernière minute pour faire valider ces points techniques : prévenez-nous dès que vous êtes prêts afin que nous puissions faire les validations de tout le monde dans les temps

Communication en cours de *sprint*

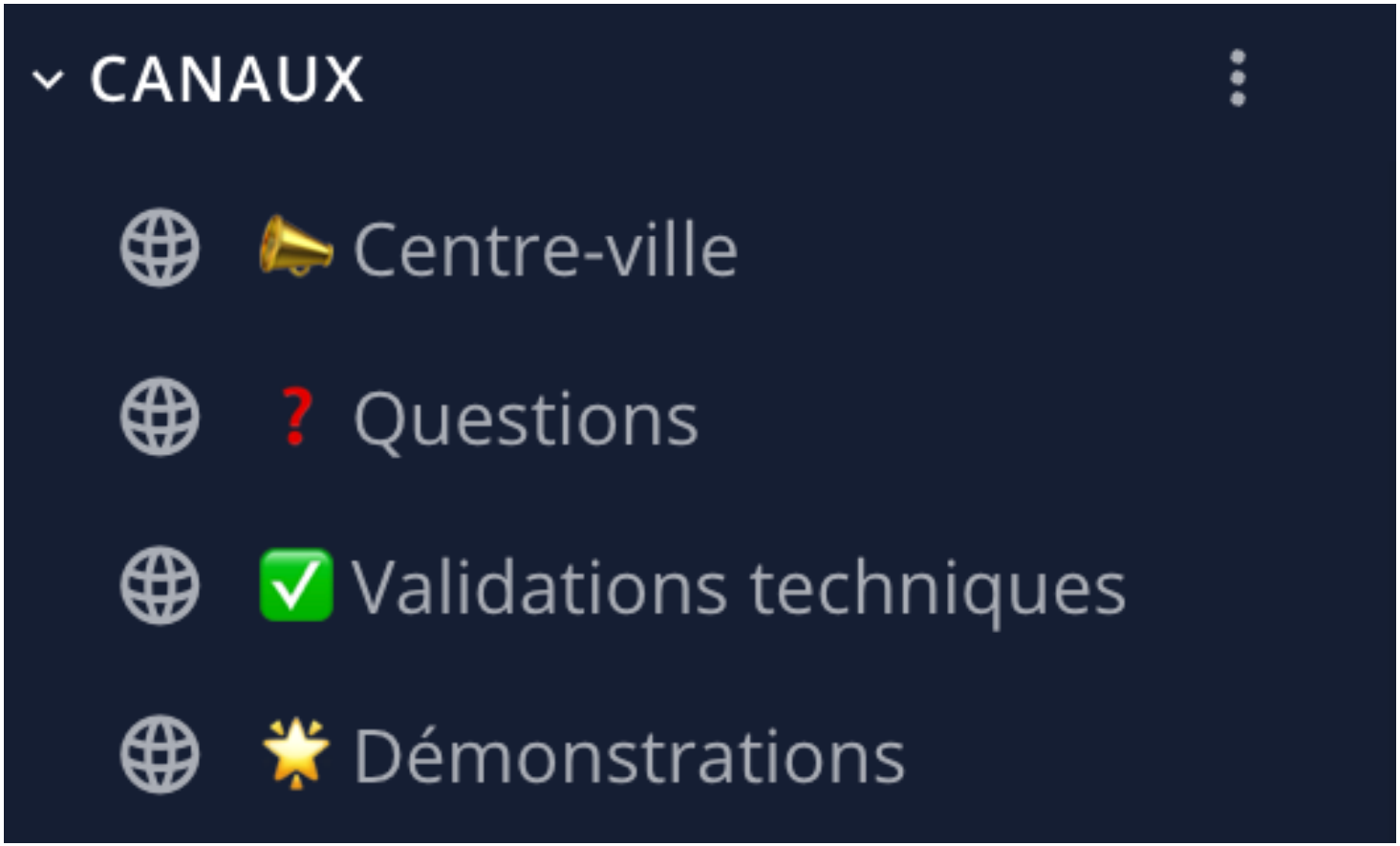


Figure – Canaux de communication

SAÉ S3.A.01 - Développement d'une Application

Sprint 1 - Représentation des nombres complexes

Romain Wallon

IUT de Lens - BUT 2 Informatique

2026-2027

UNIVERSITÉ D'ARTOIS

IUT Institut Universitaire Technologie LENS